AuroraRL

Abstract:

Nowadays, LLM Reinforcement Learning (RL) requires frequent synchronization of parameters between the trainer and distributed rollout actors. Since most organizations cannot afford tightly-coupled clusters, the alternative is loosely-coupled clusters.

Introduction:

So now we have to solve problem that comes with the loosely-coupled clusters: Capacity, Resilience, and Accessibility.

- Capacity: Post-training job requirements can exceed what any single region can supply.
- Resilience: Decoupled deployments suffer from outages, and provider-side disruptions.
- Accessibility: cross-region and cross-provider links typically offer only 1-10Gbps bandwidth and exhibit high WAN latency and loss rate.

Background:

- ### HPC Cloud Premium

  SOTA RL systems assume an RDMA-connected HPC cluster within a single datacenter as the default model. However single-datacenter HPC model introduces both cost and access barriers. RDMA clusters typically charge a network premium. For example $39/hour for 16 H100 on Hyperbolic with RDMA vs. \$31/hour without it (26% premium). Also, RL may need more than one datacenter can offer. Beyond, access remains constrained. Academic institutions, government labs, and small organizations often cannot reserve the large GPU blocks.

- ### Challenges:

  - **Challenge 1 (C1): The commodity network barrier.**

    Decentralized deployments connect the Trainer and Actors over commodity networks (1-10Gbps cross-region, cross-cloud, or heterogeneous-instance links), where full model weight transfer dominates iteration time and throttles throughput regardless of GPU speed. For example, if we have 8B model (Qwen3-8B) in BF16, pure model size will be 16GB. The transfer will take 1.3 s in an HPC fabric, within actor's 45 s generation window; 128 s on a 1Gbps link. It is beyond what even one-step asynchronous RL can hide.

- **Challenge 2 (C2): Hardware and network heterogeneity.**

  GPU instances range from older L40 and A100 GPUs to newer H100 hardware, with generation throughput differing by 2 to 3 times.
  Network conditions are equally heterogeneous: Actors from nearby providers may achieve 5 to 10 Gbps bandwidth to the Trainer, while those across continents operate at 1 to 3 Gbps.

- **Challenge 3 (C3): Dynamic membership and failures.**

  Actors can disappear involuntarily through spot preemption, hardware or network failures, and regional outages; and join or leave deliberately as operators add capacity, shift regions, or resize the pool mid-run.

- **Why do existing systems fall short?**

  - **Colocated** systems assume datacenter fabrics, homogeneous hardware, and stable membership.
  - **Disaggregated** systems tolerate some heterogeneity but require ~80Gbps dedicated links and a stable cluster.
  - **Decentralized** systems reach internet scale and absorb churn but still broadcast full weights, hiding that cost via multi-step staleness that compromises model quality.

Design Principles:

Challenges get easier to solve if we dissect the RL workflow.

- **Empirical basis: fine-grained sparsity**

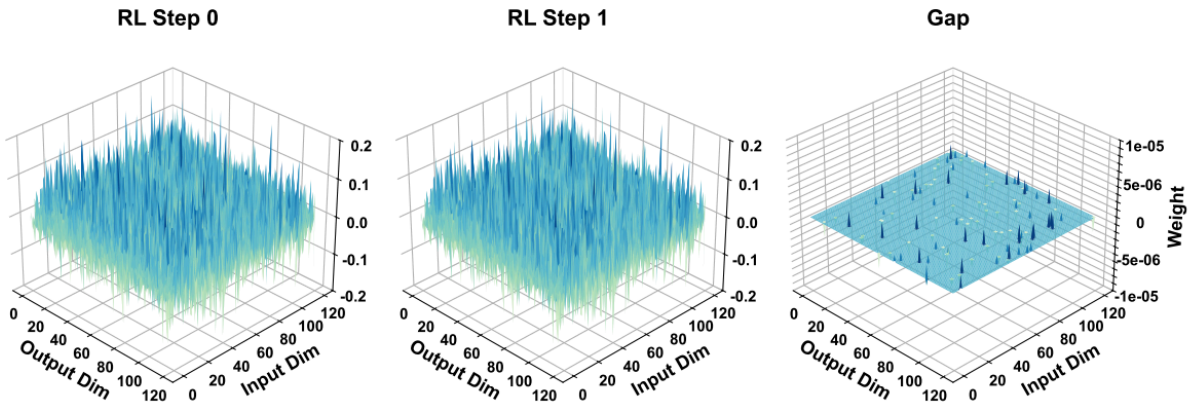
  An LLM consists of K parameter tensors $\{W^{(k)}\}_{k=1}^{K}$.
  Let $W_t^{(k)}$ and $W_{t+1}^{(k)}$ denote the values of tensor k before and after one RL step, and define the element-wise difference $\Delta W^{(k)} = W_{t+1}^{(k)} - W_t^{(k)}$ obtained by differencing consecutive checkpoints. We define the element-wise *nonzero ratio* ρ across the entire model as

$$\rho = \frac{1}{\Sigma_{k=1}^{K}|W^{(k)}|} \sum_{k=1}^{K} ||\Delta W^{(k)}||_0$$

  In simple terms it is:

$$\rho = \frac{number_of_changed_weights}{number_of_all_weights}$$

This ratio is consistently low across different model families, around 1~2%.



(a) Visualization of weight update distribution

> Visualization of weight update distribution, demonstrating the sparse nature of parameter updates.

The update ratio remains stable, rapidly falling below 1% and staying there across 800 steps. With this, no need to send all 15.6GB of weights, but only 1~2%, which is only around 202MB of data.

- ## Why sparse:

Updates are sparse across model:

Model name	Change per step
Qwen3-4B	1.12%
Llama3-8B	2.56%
GLM4-9B	1.99%
Qwen2.5-72B	1.85%

- Reason 1: RL post-training uses 10^{-6}, whereas pre-training uses 10^{-4}, meaning steps are 100 times smaller, since RL doesn't teach the model new knowledge, just reinforces.
- Reason 2: RL trains on data the model generated itself. This indistribution data naturally produces small updates, and techniques like the KL penalty and gradient clipping keep them small.
- Reason 3: The optimizer keeps its master weights in FP32, but the actors run the model in BF16. So the gap between two neighboring BF16 numbers is roughly the weight's size times 2^{-7}. For Qwen3-8B, most weights are between 10^{-3} and 10^{-1}, but most one-step updates are between 10^{-7} and 10^{-6}. Meaning, most updates are too small to change the BF16 value at all. From the actor's POV, the weight did not move.

Of all the weights that changed in FP32, only 1.27% still differ after both checkpoints are converted to BF16. That is why the sparsity ratio is measured on BF16

checkpoints: that is what the actors actually see.
This is not a temporary effect at the start of training. Over 800 steps, the ratio drops below 1%, and stays there.

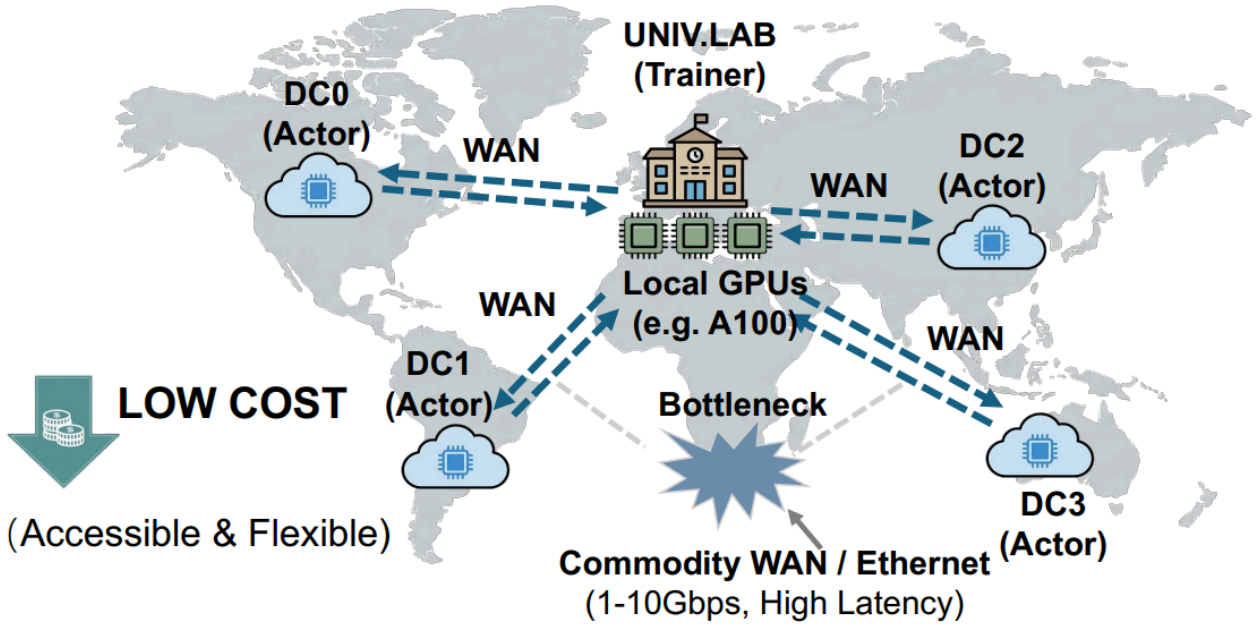
Overview of AuroraRL:

- **Deployment model:**

  The Trainer Hub connects to regional Relays over cross-cloud links up to 10 Gbps.

- **System components:**

  - The Trainer Hub runs a policy optimizer (e.g. GRPO) on a cluster (e.g. RDMA-connected H100s), maintains a Job Ledger, and operates Checkpoint Store.
  - Rollout Actors span heterogeneous hardware across geographic regions (H100, A100, and L40 clusters).



(b) Loosely coupled GPU cluster over commodity link

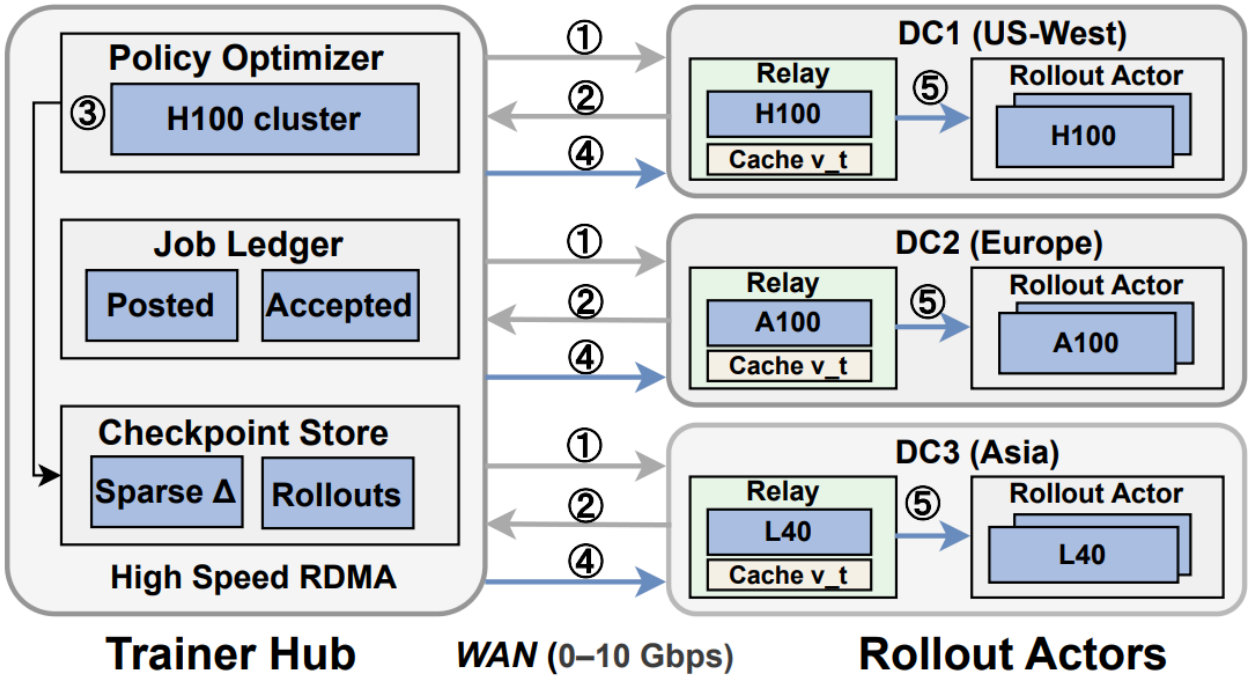
  - Relay caches the current policy version ν_t and forwards deltas to peer Actors in its region, reducing cross-region transfer from $O(N)$ to one per region.

- **Workflow:**

AuroraRL enforces strict one-step policy lag, bounding staleness while allowing rollouts, update transfer, and version preparation to overlap across regions.
There are two kinds of traffic. The gray arrows are rollout traffic: prompts going and results coming back. The blue arrows are the sparse deltas, the weight updates. On top of that, there is a control plane that coordinates jobs using time-bounded leases.

  - Step 1: The Job Ledger hands out prompts to the actors. Each actor generates rollouts using the model version it currently has active version ν.

- Step 2: The finished rollouts go back to the Checkpoint Store, where they are collected for training.
- Step 3: Inside the Trainer Hub, the policy optimizer, uses those rollouts to compute the new version $\nu + 1$.
- Step 4: The update is encoded as a sparse delta, which we call $D_{\nu+1}$. It is saved in the Checkpoint Store and sent out across the WAN, but only to the Relays, one per region.
- Step 5: Each Relay forwards the delta to the other actors in its region. The actors switch to the new version between batches, not in the middle of one.



- **IDX/VAL arrays:**

  AuroraRL stores only the nonzero, as two arrays: `idx` for the positions, and `val` for how much each one changed.

- **LEB128:**

  - **Problem:**

    If we store each index as is, then we will need 4 or 8 bytes depending on the size for indexes, and 2-byte BF16 for value. Meaning, 2/3 of data is just positions, not the actual update.

  - **Solution:**

    For example, positions `{1000, 1005, 1012, 1210}` can be stored using offset from the previous position, so that becomes `{1000, 5, 7, 198}`. Since nonzeros are evenly scattered, most gaps are small number, that can fit in a byte or two.
    We use the MSB as an indicator if there is a following byte or not. For example, 5, becomes: 0000 0101, since MSB is 0, the number is single byte size. But 198, becomes 0xC6 01. 0XC6 is 1100 0110, since MSB is 1, we need to check the second

byte, which is 1. So we decode it like 1000110 = 70 + (1 << 7) = 198.
This way, we can reduce the size by 30-50%.

- **Streaming Delta Transfer:**

  One-step lag set a deadline every iteration. Each actor must finish receiving the next delta before it finishes its current batch. If it doesn't, it either sits idle waiting, or it falls behind, and its rollout can no longer be used.
  In the paper's US-to-Canada setup with Qwen3-8B, extracting the sparce delta takes about 5 s. Then sending the 202MB delta over one TCP connection takes 4.71 s.

  - **Pipelining:**

    Instead of waiting for the whole delta file to be built, the trainer buts it into segments and sends each segment the moment it is encoded. Relays also forward each segment as soon as it arrives. So extraction, cross-region transfer, and in-region forwarding all happen at the same time. At the end, the segment are put back together in a fixed order and checked against the checkpoint's hash.

  - **Parallel TCP streams:**

    Segments are dealt out round-robin across S streams. And if one stream stalls, only its own segments wait; the other streams keep going. With this, the transfer drops from 4.71 s to 2.9 s.

  On top of both, the transfer overlaps with generation. Actors keep generating with the old version while the new delta downloads in the background. They switch only at the next safe point.

  - **Safety rule:**

    Receiving a delta (staging), is kept separate from using it (activation). When delta is fully received, the Job Ledger sends a COMMIT for version ν to the Relay. The Relay passes the same COMMIT to its peers. Then every actor switches at the end of its current batch, using an in-place scatter-add. This guarantees no rollout is ever produced by a half-updated model.

    Also, every delta is labeled with its base version, the version it must be applied on top of. An actor accepts a delta only if its current version matches that base. SO if deltas arrive out of order after a retry, they can never be applied in the wrong order.

- # Heterogeneity-aware scheduling:

  An H100 can be 2 to 3 times faster than an A100, and network bandwidth varies a lot between actors.

  - ## Problem:

If we give every actor the same amount of work, the fast ones finish early and wait for the slow ones.
Because of the one-step lag, an actor that falls two steps behind has its rollouts thrown away. So stragglers don't just slow things down; they waste work.

- **Solution:**

  - **Adaptive allocation:**

    The user sets the total batch size B for each step. The trainer keeps a throughput estimate for each actor, called τ measured in token/second. It adds up the τ of eligible actors to get T. Then each actor gets a share of the batch in proportion to its speed: $B \times \tau/T$.
    A simple example: an H100 at 5000t/s, and A100 at 2500t/s, batch of 300 requests. H100 gets 200, A100 gets 100. Both finish at the same time.

```
sum = 0
for a in actors:
    sum = a.throughput

for a in actors:
    if a.version == v or a.version == v-1:
        a.batch = math.floor(B * a.throughput / sum)

        if a.version == v-1:
            send_commit()
    else:
        a.batch = 0
        exclude(a)
```

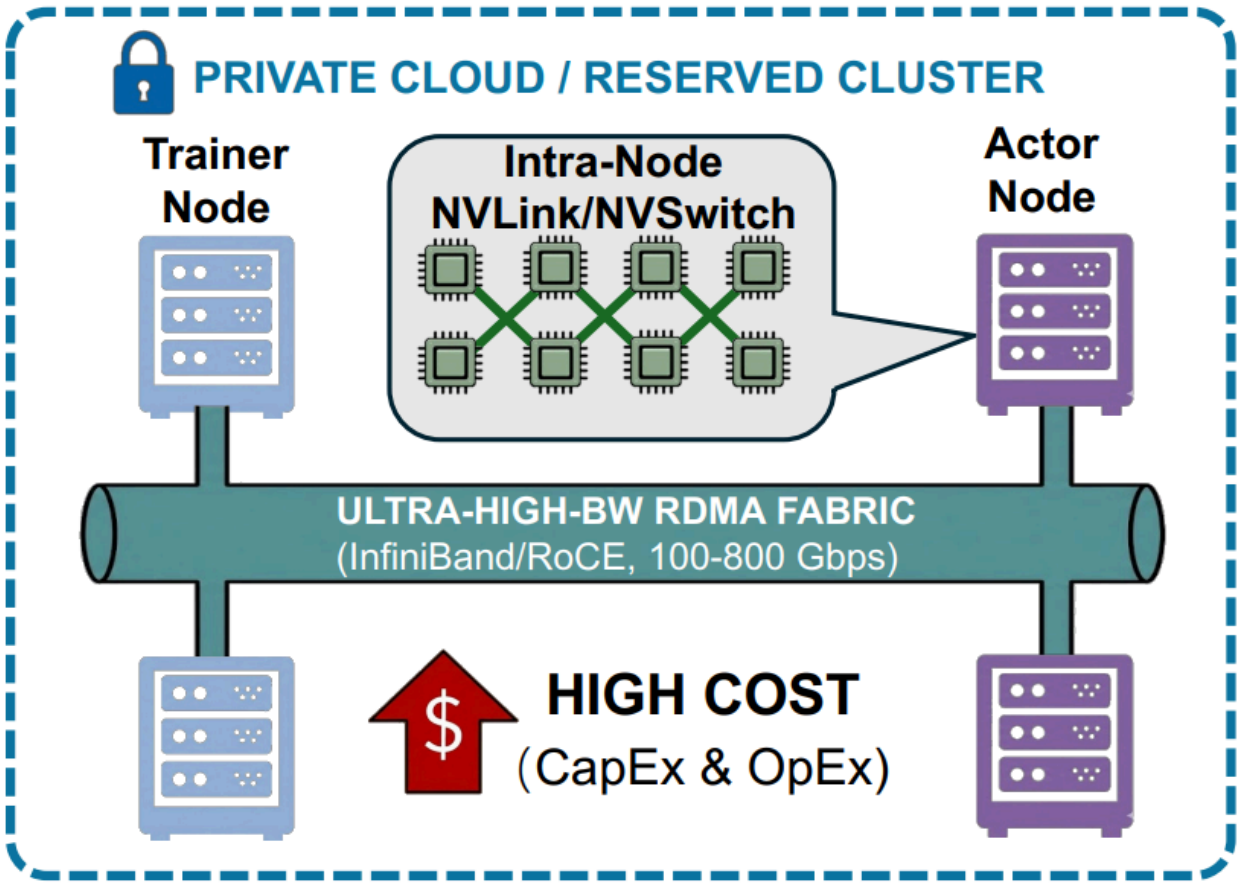
Anchors:

trainer-actor

- The Actor (The Generator):
  - Job: Runs inference to generate outputs based on the current model weights.
- The Trainer (The Learner):
  - Job: Collects the outputs and rewards generated by the actors, calculates the loss, and performs backpropagation to update the model weights.

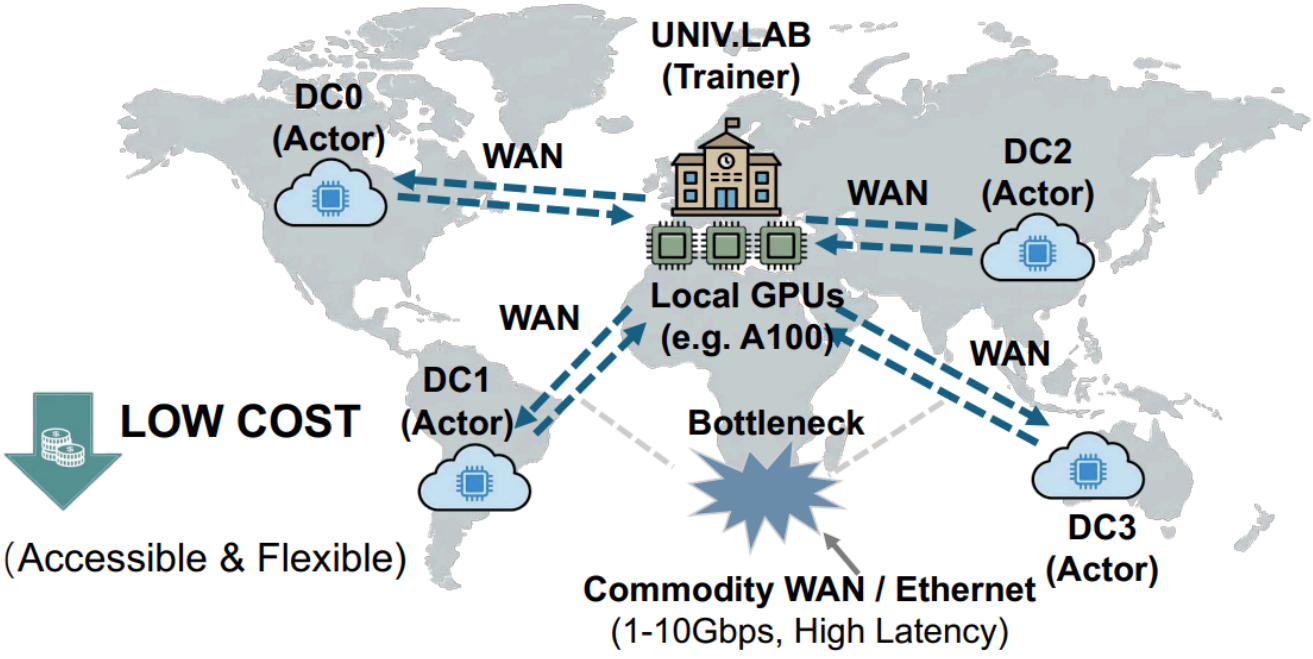
tightly-loosely-coupled:

- Tightly-Coupled cluster: System with RDMA-connected GPU HPC clusters.



(a) Tightly coupled GPU HPC cluster with RDMA

- Loosely-Coupled cluster: GPUs over standard Ethernet and WAN links.



(b) Loosely coupled GPU cluster over commodity link

rdma-hpc

In HPC (High-Performance Computing), RDMA (Remote Direct Memory Access) is a networking technology that allows computers in a cluster to r/w directly from each other's main memory without OS, CPU, or network stack.

bf16

The truncated version of float32: 1 sign bit, 8 the exponent bits, 7 mantissa bits. Unlike FP16, which has more precision, but less range, the BF16 has wider range, but less precision.

trainer-hub

The main server to update weights and distribute the deltas.

relay

A dual-role node that both generates rollouts and serves as regional proxy.

grpo

GRPO (Group Relative Policy Optimization) is a RL algorithm. Core innovation is that eliminates the memory-heavy **Critic model**, making post-training for reasoning tasks much more memory-efficient. For any given prompt q, it runs the following steps:

- i. Group sampling: The model generates a group of G different responses for the same question.
- ii. Reward Calculation: Each response is evaluated and given a reward score r_i using rule-based checks - like validating a math answer or running code - or a reward model.
- iii. Group Relative Advantage: It calculates the mean and stadnard deviation of rewards within that specific group:

$$Advantage_i = \frac{r_i - mean(r)}{std(r)}$$

- iv. Policy Update: Responses that scored above the groups average get positive advantages (the model is more likely to generate them in the future), while responses below the group average get negative advantages .

rloo

REINFORCE Leave-One-Out Instead of comparing an output to the overall groups mean (which includes the output itself), RLOO leaves the candidate output out when calculating the baseline:

$$Baseline_i = \frac{1}{G-1} \sum_{j \neq i} R_j$$

$$Advantage_i = R_i - Baseline_i$$

job-ledger

Tracks posted and accepted work.

checkpoint-store

Holds versioned sparse deltas and collected rollouts.